

Proyecto Final Libre

Curso de Programación en Python NLPY-1

Semana 9

nabllabs

2025

Objetivo del Proyecto

Desarrollar una aplicación completa en Python que integre **todos** los conceptos y técnicas aprendidas durante el curso, demostrando dominio de la programación y capacidad de resolver problemas reales.

Índice

1. Descripción General	4
1.1. Libertad Creativa	4
1.2. Salón de la Fama	4
2. Requisitos Técnicos Obligatorios	4
2.1. Semana 0: Funciones y Variables	4
2.2. Semana 1: Condicionales	5
2.3. Semana 2: Bucles	5
2.4. Semana 3: Excepciones	5
2.5. Semana 4: Librerías	5
2.6. Semana 5: Pruebas Unitarias	6
2.7. Semana 6: Entrada/Salida de Archivos	6
2.8. Semana 7: Expresiones Regulares	6
2.9. Semana 8: Programación Orientada a Objetos	6
2.10. Semana 9: Et Cetera	7
3. Evaluación	7
3.1. Distribución de Puntos	7
3.2. Bonus: Juego Retro (+20 puntos)	8
4. Ideas de Proyectos Sugeridas	8
4.1. Sistema de Gestión de Citas Médicas	8
4.2. Gestor de Finanzas Personales	9
4.3. Sistema de Biblioteca Digital	10
4.4. Administrador de Tareas y Proyectos	10
4.5. Sistema de Reservas de Hotel/Hostel	11
4.6. Plataforma de Quiz/Exámenes Online	11
4.7. Sistema de Inventario para Pulpería/Minisuper	12
4.8. Aplicación de Recetas de Cocina Tica	13
5. Entregables	13
5.1. Código Fuente	13
5.2. Documentación	14
6. Cronograma de Entrega	14
7. Recomendaciones	14
8. Criterios de Calidad del Código	15
8.1. Organización	15
8.2. Estilo	15
8.3. Documentación	15
8.4. Manejo de Errores	15
9. Ejemplos de Implementación de Requisitos	15
9.1. Ejemplo: Expresiones Regulares	15
9.2. Ejemplo: List Comprehension	16
9.3. Ejemplo: Testing	16

10.Preguntas Frecuentes	16
10.1. ¿Puedo usar librerías que no vimos en clase?	16
10.2. ¿Tengo que usar GUI o puede ser solo consola?	17
10.3. ¿Puedo trabajar en equipo?	17
10.4. ¿Qué pasa si mi proyecto no incluye algún requisito?	17
10.5. ¿Puedo hacer algo diferente a las ideas sugeridas?	17
10.6. ¿Cuánto código debo escribir?	17
11.Recursos Adicionales	17
11.1. Documentación Oficial	17
11.2. Librerías Útiles	17
12.Checklist Final	18

1. Descripción General

Este proyecto final representa la culminación de tu aprendizaje en el curso NLPY-1. A diferencia de los proyectos semanales, este proyecto es **libre**: tú decides qué aplicación desarrollar, siempre y cuando cumpla con los requisitos técnicos establecidos.

1.1. Libertad Creativa

Tienes total libertad para elegir:

- El tema de tu aplicación
- El dominio del problema a resolver
- La interfaz de usuario (consola, GUI, web)
- Las librerías adicionales que desees utilizar
- El nivel de complejidad y características extras

1.2. Salón de la Fama

¡Oportunidad Especial!

Los proyectos más **sorprendentes** y **llamativos** serán presentados en el **Salón de la Fama** de la página web de nablabs. Tu trabajo podría inspirar a futuros estudiantes y formar parte de la historia del curso.

Criterios para el Salón de la Fama:

- Originalidad y creatividad
- Calidad técnica de implementación
- Impacto visual o funcional
- Utilidad práctica
- Código limpio y bien documentado

2. Requisitos Técnicos Obligatorios

Tu proyecto **DEBE** implementar todos los temas vistos durante el curso:

2.1. Semana 0: Funciones y Variables

- Uso de funciones con parámetros y valores de retorno
- Variables de diferentes tipos (strings, integers, floats)
- Formateo de strings (f-strings)
- Métodos de strings (strip, title, capitalize, split, etc.)

- Operadores aritméticos
- Conversión de tipos (type casting)
- Función `input()` para interacción con el usuario

2.2. Semana 1: Condicionales

- Sentencias `if`, `elif`, `else`
- Operadores de comparación (`>`, `<`, `>=`, `<=`, `==`, `!=`)
- Operadores lógicos (`and`, `or`, `not`)
- Valores booleanos y expresiones booleanas
- (Opcional) Operador `match` para casos complejos

2.3. Semana 2: Bucles

- Bucles `while` para iteraciones condicionales
- Bucles `for` para iteración sobre secuencias
- Uso de `range()`
- Sentencias `break` y `continue`
- Validación de entrada con bucles

2.4. Semana 3: Excepciones

- Bloques `try` y `except`
- Manejo de múltiples tipos de excepciones
- Uso de `else` y `finally`
- Creación de excepciones personalizadas con `raise`
- Mensajes de error descriptivos

2.5. Semana 4: Librerías

- Importación de módulos estándar (`random`, `datetime`, `statistics`, etc.)
- Uso de al menos 2 librerías externas instaladas con `pip`
- Creación de al menos 1 módulo propio
- Uso apropiado de `import` y `from ... import ...`

2.6. Semana 5: Pruebas Unitarias

- Archivo separado de pruebas usando `pytest`
- Al menos 10 pruebas unitarias diferentes
- Pruebas para casos normales, casos límite y excepciones
- Uso de `assert` para validaciones
- Todas las pruebas deben pasar exitosamente

2.7. Semana 6: Entrada/Salida de Archivos

- Lectura y escritura de archivos
- Uso del bloque `with` para manejo seguro de archivos
- Trabajo con archivos CSV (usando el módulo `csv`)
- Persistencia de datos entre ejecuciones
- Manejo de errores en operaciones de archivos

2.8. Semana 7: Expresiones Regulares

- Uso del módulo `re`
- Al menos 3 patrones regex diferentes
- Validación de formatos (emails, teléfonos, cédulas, etc.)
- Uso de funciones como `search()`, `match()`, `findall()`, o `sub()`
- Patrones con metacaracteres y cuantificadores

2.9. Semana 8: Programación Orientada a Objetos

- Al menos 4 clases diferentes bien diseñadas
- Uso de `__init__` (constructor)
- Atributos de instancia y métodos
- Al menos 1 ejemplo de herencia
- Uso de `@property` para encapsulación
- Métodos especiales (`__str__`, `__repr__`)
- (Opcional) `@classmethod` o `@staticmethod`

2.10. Semana 9: Et Cetera

- Uso de conjuntos (sets) o diccionarios avanzados
- Al menos 2 list comprehensions
- Al menos 1 dictionary comprehension
- Uso de funciones lambda
- (Opcional) Uso de `*args` o `**kwargs`
- (Opcional) Decoradores o generadores

3. Evaluación

3.1. Distribución de Puntos

Criterio	Puntos
Implementación de todos los requisitos técnicos	40
Funcionalidad y lógica del programa	20
Calidad del código (limpieza, organización)	15
Documentación y comentarios	10
Pruebas unitarias completas y efectivas	10
Creatividad e innovación	5
Total	100
Bonus: Juego Retro	+20

3.2. Bonus: Juego Retro (+20 puntos)

Desafío Especial: Juegos Retro

Si decides desarrollar un **juego retro clásico**, recibirás **20 puntos adicionales** por la complejidad técnica que esto implica.

Ejemplos de juegos retro:

- Space Invaders
- Bomberman (estilo NES)
- Donkey Kong
- Snake
- Pac-Man
- Asteroids
- Pong
- Tetris

Requisitos adicionales para el bonus:

- El juego debe ser jugable y funcional
- Debe incluir mecánicas básicas del juego original
- Interfaz gráfica (puede usar `pygame`, `tkinter`, etc.)
- Sistema de puntuación
- **IMPORTANTE:** Debe implementar TODOS los temas del curso

Nota: Desarrollar un juego retro es significativamente más complejo, pero también más gratificante y tiene mayor probabilidad de llegar al Salón de la Fama.

4. Ideas de Proyectos Sugeridas

A continuación se presentan 8 ideas de proyectos completamente desarrolladas. Puedes elegir una de estas o crear tu propia idea original.

4.1. Sistema de Gestión de Citas Médicas

Descripción: Aplicación para gestionar citas médicas en una clínica u hospital.

Componentes principales:

- **Clases:** Paciente, Doctor, Cita, Clínica
- **GUI:** Formularios para registrar pacientes/doctores, calendario visual de citas

- **Archivos:** Guardar datos en CSV/JSON, exportar reportes de citas
- **Regex:** Validar cédulas costarricenses, emails, teléfonos
- **Librerías:** `datetime` para manejo de fechas y horarios, `random` para generar códigos únicos de cita
- **Testing:** Validar lógica de reservas, detectar conflictos de horarios

Funcionalidades:

- Registro de pacientes y doctores
- Agendar, modificar y cancelar citas
- Validar disponibilidad de horarios
- Búsqueda de citas por paciente, doctor o fecha
- Recordatorios de citas próximas
- Generación de reportes estadísticos

4.2. Gestor de Finanzas Personales

Descripción: Aplicación para controlar ingresos, gastos y presupuestos personales.

Componentes principales:

- **Clases:** Cuenta, Transacción, Categoría, Usuario
- **GUI:** Dashboard con gráficos de gastos, formularios de ingreso/egreso
- **Archivos:** Persistencia de transacciones, importar extractos bancarios CSV
- **Regex:** Validar números de cuenta, parsear descripciones de transacciones
- **Librerías:** `matplotlib` para gráficos, `statistics` para análisis
- **Excepciones:** Manejo de saldos insuficientes, transacciones inválidas

Funcionalidades:

- Registro de ingresos y gastos
- Categorización de transacciones
- Visualización de gastos por categoría
- Presupuestos mensuales con alertas
- Reportes financieros con gráficos
- Importación de datos desde archivos CSV

4.3. Sistema de Biblioteca Digital

Descripción: Plataforma para gestionar préstamos de libros en una biblioteca.

Componentes principales:

- **Clases:** Libro, Usuario, Préstamo, Biblioteca
- **GUI:** Catálogo visual, búsqueda avanzada, gestión de préstamos
- **Archivos:** Base de datos de libros en CSV, historial de préstamos
- **Regex:** Búsqueda por patrones (ISBN, autor, título)
- **Librerías:** `datetime` para fechas de devolución, `random` para códigos
- **Testing:** Validar reglas de préstamo, cálculo de multas por atraso

Funcionalidades:

- Catálogo de libros con búsqueda avanzada
- Sistema de préstamos y devoluciones
- Control de disponibilidad
- Cálculo automático de multas
- Historial de préstamos por usuario
- Reservas de libros prestados

4.4. Administrador de Tareas y Proyectos

Descripción: Aplicación tipo Trello para gestionar tareas y proyectos.

Componentes principales:

- **Clases:** Tarea, Proyecto, Etiqueta, Usuario
- **GUI:** Tableros visuales, filtros, calendario de tareas
- **Archivos:** Guardar proyectos y tareas, exportar reportes
- **Regex:** Parsear fechas, buscar por etiquetas
- **Librerías:** `datetime`, `os` para organizar archivos
- **Comprehensions:** Filtrar tareas por estado, prioridad, fecha

Funcionalidades:

- Creación de proyectos y tareas
- Estados de tareas (por hacer, en progreso, completado)
- Sistema de prioridades

- Fechas límite con recordatorios
- Etiquetas y categorías personalizadas
- Vista de calendario
- Exportación de reportes de productividad

4.5. Sistema de Reservas de Hotel/Hostel

Descripción: Plataforma para gestionar reservas de habitaciones.

Componentes principales:

- **Clases:** Habitación, Reserva, Cliente, Hotel
- **GUI:** Calendario de disponibilidad, formulario de reserva
- **Archivos:** Persistencia de reservas, generación de facturas
- **Regex:** Validar pasaportes, tarjetas de crédito (formato)
- **Librerías:** `datetime` para fechas, `random` para códigos de reserva
- **Excepciones:** Manejo de overbooking, cancelaciones

Funcionalidades:

- Búsqueda de habitaciones disponibles
- Sistema de reservas con confirmación
- Diferentes tipos de habitaciones y tarifas
- Check-in y check-out
- Generación de facturas
- Cancelaciones y reembolsos
- Estadísticas de ocupación

4.6. Plataforma de Quiz/Exámenes Online

Descripción: Sistema para crear y realizar exámenes en línea.

Componentes principales:

- **Clases:** Pregunta, Examen, Estudiante, Resultado
- **GUI:** Interfaz de examen, timer, resultados con gráficos
- **Archivos:** Banco de preguntas en CSV, exportar calificaciones
- **Regex:** Validar formatos de respuestas específicas
- **Librerías:** `random` para barajar preguntas, `statistics` para análisis

- **Testing:** Verificar lógica de calificación automática

Funcionalidades:

- Creación de bancos de preguntas
- Diferentes tipos de preguntas (opción múltiple, verdadero/falso, respuesta corta)
- Temporizador para exámenes
- Calificación automática
- Barajeo de preguntas y opciones
- Estadísticas de desempeño
- Exportación de resultados

4.7. Sistema de Inventario para Pulpería/Minisuper

Descripción: Aplicación de punto de venta e inventario para negocio.

Componentes principales:

- **Clases:** Producto, Venta, Cliente, Inventario
- **GUI:** Punto de venta, gestión de stock, reportes visuales
- **Archivos:** Base de datos de productos, registro de ventas
- **Regex:** Validar códigos de barras, buscar productos
- **Librerías:** `datetime` para ventas, `statistics` para análisis
- **Excepciones:** Stock insuficiente, precios inválidos

Funcionalidades:

- Registro de productos con código de barras
- Control de inventario en tiempo real
- Proceso de venta con cálculo de total
- Alertas de stock bajo
- Historial de ventas
- Reportes de productos más vendidos
- Gestión de proveedores

4.8. Aplicación de Recetas de Cocina Tica

Descripción: Plataforma para gestionar y buscar recetas de comida costarricense.

Componentes principales:

- **Clases:** Receta, Ingrediente, Usuario, Favoritos
- **GUI:** Buscador visual, filtros por ingredientes, timer de cocina
- **Archivos:** Base de datos de recetas, exportar lista de compras
- **Regex:** Parsear cantidades (“2 tazas”, “500g”), buscar ingredientes
- **Librerías:** random para sugerir recetas del día
- **Comprehensions:** Filtrar recetas por ingredientes disponibles

Funcionalidades:

- Base de datos de recetas típicas costarricenses
- Búsqueda por nombre, ingredientes o categoría
- Sistema de favoritos
- Generación automática de lista de compras
- Conversión de unidades
- Timer para cocinar
- Sugerencias aleatorias de recetas
- Exportar recetas a archivo

5. Entregables

5.1. Código Fuente

- Todos los archivos `.py` necesarios
- Módulos organizados en carpetas apropiadas
- Archivo de pruebas `test_*.py`
- Archivos de datos (CSV, JSON, etc.)
- Archivo `requirements.txt` con todas las dependencias

5.2. Documentación

- README.md con:
 - Descripción del proyecto
 - Instrucciones de instalación
 - Cómo ejecutar el programa
 - Cómo ejecutar las pruebas
 - Ejemplos de uso
- Comentarios en el código explicando lógica compleja
- Docstrings en todas las funciones y clases

6. Cronograma de Entrega

Fecha	Entregable
Semana 9, Día 3	Propuesta de proyecto (1 párrafo)
Semana 9, Día 5	Diseño de clases y estructura
Semana 10, Día 3	Avance del 50 % funcional
Semana 10, Día 7	Entrega final completa

7. Recomendaciones

1. **Planifica primero:** Diseña tu aplicación antes de programar. Define las clases, funciones y flujo de datos.
2. **Desarrollo iterativo:** No intentes hacer todo de una vez. Desarrolla por características incrementales.
3. **Prueba continuamente:** Escribe pruebas mientras desarrollas, no al final.
4. **Commits frecuentes:** Haz commits pequeños y frecuentes con mensajes descriptivos.
5. **Código limpio:** Sigue convenciones de estilo (PEP 8), usa nombres descriptivos, evita repetición.
6. **Maneja errores:** No dejes que tu programa crashee. Maneja todas las excepciones apropiadamente.
7. **Documenta:** Escribe docstrings y comentarios útiles. Tu código debe ser entendible por otros.
8. **Sé creativo:** Los proyectos más memorables son los que muestran originalidad e innovación.
9. **Piensa en el usuario:** Haz que tu aplicación sea intuitiva y fácil de usar.
10. **Ve más allá:** Los requisitos son el mínimo. Agregar características extra te distinguirá.

8. Criterios de Calidad del Código

8.1. Organización

- Estructura de archivos lógica y clara
- Separación de responsabilidades (clases bien definidas)
- Módulos importados correctamente

8.2. Estilo

- Cumplimiento con PEP 8
- Nombres de variables descriptivos (no `x`, `y`, `data`)
- Funciones con un propósito claro y único
- Longitud apropiada de funciones (no más de 50 líneas idealmente)

8.3. Documentación

- Docstrings en todas las clases y funciones públicas
- Comentarios explicando lógica no obvia
- README completo y claro

8.4. Manejo de Errores

- Excepciones manejadas apropiadamente
- Mensajes de error útiles para el usuario
- No uso de `except:` sin especificar el tipo de excepción

9. Ejemplos de Implementación de Requisitos

9.1. Ejemplo: Expresiones Regulares

```
import re

class ValidadorCostarricense:
    """Validador de formatos costarricenses"""

    @staticmethod
    def validar_cedula(cedula):
        """Valida formato de cédula costarricense (0-0000-0000)"""
        patron = r'^\d-\d{4}-\d{4}$'
        return re.match(patron, cedula) is not None
```

```
@staticmethod
def validar_telefono(telefono):
    """Valida teléfono costarricense (8 dígitos)"""
    patron = r'^[2-8]\d{3}-?\d{4}$'
    return re.match(patron, telefono) is not None
```

9.2. Ejemplo: List Comprehension

```
# Filtrar tareas pendientes con alta prioridad
tareas_urgentes = [tarea for tarea in self.tareas
                    if tarea.prioridad == 'alta'
                    and tarea.estado == 'pendiente']

# Obtener nombres de clientes activos
nombres_activos = [cliente.nombre.title()
                   for cliente in self.clientes
                   if cliente.activo]
```

9.3. Ejemplo: Testing

```
import pytest
from sistema import Cuenta, TransaccionInvalida

def test_deposito_valido():
    cuenta = Cuenta("Juan", 1000)
    cuenta.depositar(500)
    assert cuenta.saldo == 1500

def test_retiro_saldo_insuficiente():
    cuenta = Cuenta("María", 100)
    with pytest.raises(TransaccionInvalida):
        cuenta.retirar(200)

def test_transferencia_exitosa():
    cuenta1 = Cuenta("Pedro", 1000)
    cuenta2 = Cuenta("Ana", 500)
    cuenta1.transferir(cuenta2, 300)
    assert cuenta1.saldo == 700
    assert cuenta2.saldo == 800
```

10. Preguntas Frecuentes

10.1. ¿Puedo usar librerías que no vimos en clase?

Sí, de hecho es recomendado. Debes usar al menos 2 librerías externas, y puedes elegir las que mejor se adapten a tu proyecto.

10.2. ¿Tengo que usar GUI o puede ser solo consola?

Puedes usar consola, GUI o incluso web. Lo importante es la funcionalidad y el cumplimiento de requisitos técnicos.

10.3. ¿Puedo trabajar en equipo?

Este proyecto es individual. Sin embargo, puedes discutir ideas y conceptos con compañeros, siempre que el código sea tuyo.

10.4. ¿Qué pasa si mi proyecto no incluye algún requisito?

Cada requisito faltante resultará en deducción de puntos. Asegúrate de implementar TODO.

10.5. ¿Puedo hacer algo diferente a las ideas sugeridas?

¡Por supuesto! Las ideas son solo sugerencias. Tu creatividad es bienvenida y valorada.

10.6. ¿Cuánto código debo escribir?

No hay un mínimo estricto, pero un proyecto completo típicamente tiene 500-1500 líneas de código (sin contar archivos de datos).

11. Recursos Adicionales

11.1. Documentación Oficial

- Python: <https://docs.python.org/3/>
- Pytest: <https://docs.pytest.org/>
- Expresiones Regulares: <https://docs.python.org/3/library/re.html>

11.2. Librerías Útiles

- `tkinter` - GUI básica
- `pygame` - Juegos 2D
- `matplotlib` - Gráficos
- `pandas` - Manipulación de datos
- `requests` - APIs HTTP
- `pillow` - Procesamiento de imágenes

12. Checklist Final

Antes de entregar, verifica que tu proyecto incluya:

Funciones con parámetros y return

Variables y type casting

F-strings y métodos de strings

Condicionales (if/elif/else)

Operadores lógicos

Bucles while y for

Break y continue

Try/except con al menos 3 tipos de excepciones

Raise para excepciones personalizadas

Importación de módulos estándar

Al menos 2 librerías externas

Creación de módulo propio

Archivo de pruebas con pytest

Al menos 10 pruebas unitarias

Lectura y escritura de archivos

Uso de archivos CSV

Bloque with para archivos

Al menos 3 patrones regex

Funciones del módulo re

Al menos 4 clases

Uso de `__init__`

Al menos 1 ejemplo de herencia

`@property`

`__str__` y/o `__repr__`

Al menos 2 list comprehensions

Al menos 1 dictionary comprehension

Funciones lambda

Sets o diccionarios avanzados

README.md completo

requirements.txt

Código comentado y documentado

Todas las pruebas pasan

¡Mucho Éxito!

Este proyecto es tu oportunidad para demostrar todo lo que has aprendido. No tengas miedo de ser ambicioso y creativo. Los mejores proyectos son aquellos que resuelven problemas reales o que simplemente son divertidos de usar.

Recuerda: El objetivo no es solo cumplir requisitos, sino crear algo de lo que estés orgulloso.

¡Nos vemos en el Salón de la Fama!